TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

HIMALAYA COLLEGE OF ENGINEERING

[CODE: ENCT 354]

A PROJECT PROPOSAL REPORT

ON

AN INTELLIGENT ADAPTIVE MULTI-VENDOR ELECTRONICS MARKETPLACE WEB APPLICATION

Submitted By

Aastha Mainali (HCE080BCT001)

Aayushma Ghale (HCE080BCT003)

Anjila Pandey (HCE080BCT007)

Anshu Regmi (HCE080BCT009)

A Project Proposal Submitted to Department of Electronics and Computer Engineering in Partial Fulfillment of the Requirement for Bachelor's Degree in Computer Engineering

Department of Electronics and Computer Engineering

Lalitpur, Nepal

June, 2026

NEXORA: AN INTELLIGENT ADAPTIVE MULTI-VENDOR ELECTRONICS MARKETPLACE WEB APPLICATION

Submitted By

Aastha Mainali (HCE080BCT001)

Aayushma Ghale (HCE080BCT003)

Anjila Pandey (HCE080BCT007)

Anshu Regmi (HCE080BCT009)

A Project Report Submitted in Partial Fulfillment for the Requirement of the
Bachelor's Degree in Computer Engineering

Department of Electronics and Computer Engineering

HIMALAYA COLLEGE OF ENGINEERING

Tribhuvan University

Lalitpur, Nepal

June, 2026

ABSTRACT

This proposal presents the design of an Intelligent Adaptive Multi-Vendor Electronics Marketplace Web Application, a platform that enables customers to discover, compare, and purchase electronics products from multiple verified sellers. The system integrates an artificial-intelligence-driven recommendation engine based on cosine similarity, a custom-built conversational shopping assistant powered by prompt engineering over a large language model, an OpenCV-powered visual image search engine, and a unified semantic search and price-filtering pipeline. The backend is built on Django REST Framework and the frontend on React.js, following a RESTful single-page application architecture. A MySQL database manages all persistent data including user accounts, product listings, orders, and reviews. The platform accommodates three user roles customer, seller, and administrator with role-based access control throughout. The expected outcome is a functional, intelligent, and user-friendly marketplace that overcomes the personalization and discovery limitations of conventional electronics retail platforms.

Keywords: *e-commerce, electronics, marketplace, recommendation system, prompt engineering, conversational assistant, visual search, OpenCV, web application*

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
B2C	Business-to-Consumer
CBF	Content-Based Filtering
CF	Collaborative Filtering
CRUD	Create, Read, Update, Delete
CV	Computer Vision
DBMS	Database Management System
DFD	Data Flow Diagram
DRF	Django REST Framework
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MVC	Model-View-Controller
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PE	Prompt Engineering
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience

CHAPTER 1:
INTRODUCTION

1.1 Background Information

The rapid growth of internet connectivity and mobile device adoption has fundamentally shifted consumer purchasing behaviour from traditional brick-and-mortar retail to digital platforms. Global e-commerce revenue has grown consistently over the past decade, with electronics representing one of the largest and fastest-expanding product categories. In South Asian markets such as Nepal, platforms like Daraz have demonstrated strong consumer demand for online electronics purchasing; however, these platforms offer limited personalization, generic search experiences, and no intelligent assistance to guide purchasing decisions.

Simultaneously, advances in artificial intelligence particularly in machine learning-based recommendation systems and large language model-driven conversational agents have created practical opportunities to embed intelligence directly into the retail experience. Recommendation engines can suggest relevant products based on user preferences and item attributes, while natural language interfaces allow users to describe their needs conversationally rather than navigating complex filter menus.

The proposed system is a web-based, adaptive multi-vendor electronics marketplace that integrates these capabilities into a unified platform. It is built on a React.js frontend communicating with a Django REST Framework backend, uses MySQL for persistent storage, and incorporates a content-based cosine similarity recommendation engine alongside a custom conversational shopping assistant developed through prompt engineering techniques applied to a large language model. The system supports three user roles customer, seller, and administrator each with a dedicated interface and access control layer.

1.2 Problem Statement

Existing electronics e-commerce platforms in the Nepali and broader South Asian market face several significant limitations:

- **Keyword-dependent product discovery:** Users must know exact product names, brands, or model numbers to find relevant items, making product search difficult for many customers.
- **Lack of personalization:** Most platforms display the same product catalog to all users regardless of their browsing history, budget, or preferences, resulting in a less relevant shopping experience.
- **Limited conversational assistance:** Available chatbot systems primarily focus on post-purchase services such as order tracking and customer support rather than assisting users in selecting suitable products before purchase.
- **Disconnected search and filtering mechanisms:** Search functionality, category navigation, and price filters often operate independently, reducing the effectiveness of product discovery and recommendation systems.
- **Poor seller verification and management:** Smaller platforms frequently lack robust seller onboarding and verification processes, leading to unreliable listings, inconsistent product quality, and reduced customer trust.
- **Unsatisfactory user experience:** These limitations contribute to poor product discovery, especially for technically complex electronics, higher cart abandonment rates, and lower customer satisfaction among users who are unfamiliar with technical specifications.

1.3 Objectives

The specific objectives of this project are:

- To design and develop a full-stack multi-vendor electronics marketplace web application with role-based access control for customers, sellers, and administrators.
- To implement seller verification, product listing management, order processing, and order tracking functionalities to ensure a reliable marketplace environment.

- To develop an advanced product discovery system integrating semantic vector search, price-range filtering, category-based navigation, and OpenCV-powered visual image search.
- To build a content-based recommendation engine using cosine similarity over product attribute vectors to provide personalized product suggestions.
- To develop a conversational shopping assistant using prompt engineering and large language models for natural language query handling and product guidance.
- To evaluate the usability, recommendation quality, and overall performance of the system in enhancing the online electronics shopping experience.

1.4 Scope and Applications

The scope of this project encompasses the full-stack development of a web-based marketplace platform targeting the electronics retail domain. The system will support product categories including mobile phones, laptops, tablets, accessories, and consumer electronics. The platform will be accessible via modern web browsers and will not require a dedicated mobile application at the initial release.

The key functional scope includes: user registration and authentication for all three roles; product listing, editing, and management by verified sellers; browsing, searching, filtering, and purchasing by customers; AI-powered recommendations and conversational assistance; visual image-based product search powered by OpenCV; order placement and status tracking; a review and rating system; and an administrative dashboard for user and content management.

The system is applicable to small and medium electronics retailers in Nepal seeking a digital presence, consumers seeking guided product discovery, and academic demonstration of AI integration in full-stack web applications.

CHAPTER 2:
LITERATURE REVIEW

This chapter reviews existing research and theoretical foundations relevant to the proposed system, organized across four domains: e-commerce platform architecture, AI-based recommendation systems, conversational AI and shopping assistants, and product search and filtering. Each section evaluates prior work, identifies limitations, and establishes how the proposed system addresses them.

2.1 E-Commerce Platforms and System Architecture

E-commerce has fundamentally transformed global retail by enabling buyers and sellers to interact through digital platforms without geographical constraints. This paper defines electronic commerce as the process of buying, selling, or exchanging products, services, and information via computer networks, and identifies multi-role user systems comprising customers, sellers, and administrators as a core architectural requirement of modern marketplace platforms [1]. This three-actor model forms the structural basis of the proposed system.

This paper further categorizes e-commerce platforms by their business model, distinguishing between business-to-consumer (B2C) and marketplace models where third-party sellers list and manage their own inventory [2]. The proposed system adopts this marketplace model, requiring a dedicated seller registration and verification workflow, an administrative control panel, and role-based access control to ensure system integrity.

From an architectural standpoint, this paper advocates for the separation of frontend and backend concerns through RESTful API design, which enables scalable, loosely coupled systems capable of independently evolving their client and server components [3]. The proposed system implements this principle by combining a React.js single-page application (SPA) on the frontend with a Django REST Framework (DRF) backend, communicating exclusively through JSON-based API endpoints. This architectural choice is further validated by another paper, which argues that RESTful APIs provide a standardized, stateless communication model suitable for applications that must serve multiple client types in this case, customer, seller, and administrator interfaces from a

single backend [4].

Despite the maturity of existing e-commerce platforms such as Amazon and Daraz, the majority of conventional systems present the same product catalog uniformly to all users, ignoring individual preferences, browsing history, and behavioral patterns. The proposed system addresses this gap by integrating an AI-driven personalization layer directly into the core architecture, enabling intelligent and adaptive user experiences rather than static product displays.

2.2 AI-Based Recommendation Systems

Product recommendation systems have become one of the most commercially impactful applications of machine learning in the e-commerce domain. This paper introduces item-to-item collaborative filtering at Amazon, demonstrating that recommendations derived from user behavior and purchase history could substantially increase product discovery and sales conversion [5]. Their work established the foundation for recommendation system research.

This paper provides a comprehensive taxonomy of recommendation approaches, classifying them into collaborative filtering (CF), content-based filtering (CBF), and hybrid methods [6]. Collaborative filtering relies on user interactions, while content-based filtering focuses on product attributes. Hybrid methods combine both approaches.

The proposed system implements a content-based filtering approach using Cosine Similarity. This design choice is deliberately suited to the cold-start problem because collaborative filtering requires large historical datasets. Content-based filtering allows recommendations to be generated from the first interaction without dependency on user history. This paper confirms that such methods are effective in sparse data environments [7].

This paper surveys deep learning approaches to recommendation systems, including neural models and sequence-based recommendation techniques [8]. While powerful, these approaches require large datasets and high computational resources. For this system, a lightweight content-based model provides a better balance of performance and feasibility.

2.3 Conversational AI, Prompt Engineering, and Shopping Assistants

The integration of conversational agents into e-commerce is an important evolution in user interaction. This paper evaluates chatbot systems and finds that rule-based chatbots perform well only for simple queries but fail in complex, flexible conversations [9].

This paper introduces large language models and demonstrates their ability to perform tasks through few-shot learning without task-specific training [10]. Instead of directly using raw LLM outputs, the proposed system uses prompt engineering.

This paper defines prompt engineering as the structured design of prompts including role instructions, domain context, and output constraints to guide model behavior [11]. This allows control over assistant behavior without retraining.

This paper shows that structured prompting improves accuracy and consistency in NLP tasks [12]. In the proposed system, prompts are carefully designed to embed product context and constraints, turning the model into a domain-specific shopping assistant.

This paper studies slot-filling techniques in conversational systems, where structured information such as product type and price range is extracted from user queries [13]. The proposed system uses similar principles to convert natural language into structured search inputs.

Compared to existing chatbot systems in e-commerce, the proposed assistant supports full product discovery rather than just support queries.

2.4 Product Search, Price Filtering, Visual Search, and Information Retrieval

This paper establishes vector-based information retrieval models for matching queries with relevant documents [14]. The proposed system extends this concept for product search using attribute vectors.

This paper demonstrates that faceted search improves user experience by allowing multiple filters such as category, brand, and price [15]. The proposed system incorporates this idea into its search engine.

This paper explains the role of OpenCV in computer vision tasks such as feature extraction and image comparison [17]. The proposed system uses similar techniques for visual product search by comparing uploaded images with stored product features.

This paper also emphasizes that navigation and labeling structures significantly affect usability [16]. The proposed system integrates search, filtering, and visual search into a unified interface.

2.5 Summary and Identified Research Gaps

Across the literature, individual components such as recommendation systems, chatbots, and visual search have been studied independently. However, their integration into a unified intelligent e-commerce system is still limited.

(i) Cold-start recommendation is addressed using content-based methods [7].

(ii) Conversational AI is improved using prompt engineering techniques [10, 11, 12].

(iii) Visual search using computer vision remains underused in smaller e-commerce platforms [17].

(iv) Search and filtering are often implemented as separate systems rather than unified pipelines [14, 15].

(v) Electronics-specific personalization is still limited in general platforms [6].

The proposed system integrates all these components into a unified architecture for improved user experience in electronics e-commerce.

CHAPTER 3:
REQUIREMENT ANALYSIS AND SYSTEM DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

The functional requirements define the core behaviours the system must support for each user role.

- **Customer:** Register and log in; browse, search, and filter products by category, brand, price range, and specifications; perform visual image-based product search by uploading a photo of an electronics item; view product details and specifications; add products to cart and wishlist; place and track orders; write product reviews; receive AI-powered product recommendations; interact with the conversational shopping assistant.
- **Seller:** Register as a seller and undergo verification; list, edit, and delete products with images, descriptions, pricing, and stock levels; manage incoming orders and update fulfillment status; view sales analytics on the seller dashboard.
- **Administrator:** Approve or reject seller verification requests; manage all users, products, categories, and orders; monitor platform activity via an administrative dashboard; manage product categories and brand listings.

3.1.2 Non-Functional Requirements

- **Performance:** API response time must not exceed 2 seconds for standard product queries; recommendation results must be generated within 3 seconds.
- **Scalability:** The system architecture must support horizontal scaling of the backend to accommodate increasing product catalog size and concurrent user load.
- **Security:** All authentication must use JWT-based tokens; passwords must be stored using bcrypt hashing; API endpoints must enforce role-based access control.

- **Usability:** The interface must be intuitive for non-technical users; the search and filter interface must require no prior knowledge of product model numbers.
- **Reliability:** The system must achieve at least 99% uptime during evaluation; order data must be persisted with transactional integrity.
- **Maintainability:** Code must follow modular architecture patterns; the backend must use Django ORM for all database interactions to ensure portability.

3.2 Feasibility Analysis

Technical Feasibility: All technologies required for the proposed system React.js, Django REST Framework, MySQL, Scikit-learn, and the LLM inference endpoint are mature, well-documented, and freely available or accessible at low cost. The development team has prior experience with Python and JavaScript, and the required libraries impose no significant learning curve beyond the prompt engineering and AI integration components.

Operational Feasibility: The system targets a clearly defined user base (online electronics buyers and small-to-medium electronics sellers in Nepal) with demonstrated demand. The three-role architecture maps directly onto the operational workflows of a marketplace.

Economic Feasibility: The project incurs effectively zero monetary cost. All required frameworks and libraries are open-source. The conversational assistant uses a free-tier LLM inference endpoint (such as Groq API or a locally hosted Ollama instance), incurring no API charges. Deployment is handled through free-tier cloud hosting services. Internet access is provided through college infrastructure, and the .np domain and SSL certificate are both available at no cost. The only anticipated expense is printing and documentation for the final submission.

Schedule Feasibility: The project is planned over one academic semester using an Agile SCRUM-based methodology, with four defined sprints aligned to the major functional modules. The schedule is detailed in Chapter 5.

3.3 System Design

3.3.1 Context Diagram

The context diagram represents the overall interaction between the Intelligent Multi-Vendor Electronics Marketplace System and its three primary external entities: the Customer, the Seller, and the Administrator. The Customer interacts with the system to browse products, place orders, and receive recommendations. The Seller interacts to list products and manage inventory. The Administrator interacts to approve sellers, manage content, and oversee platform operations. External services the LLM inference endpoint and the Payment Gateway (Stripe/Razorpay) interface with the system as third-party components.

3.3.2 Data Flow Diagram Level 0

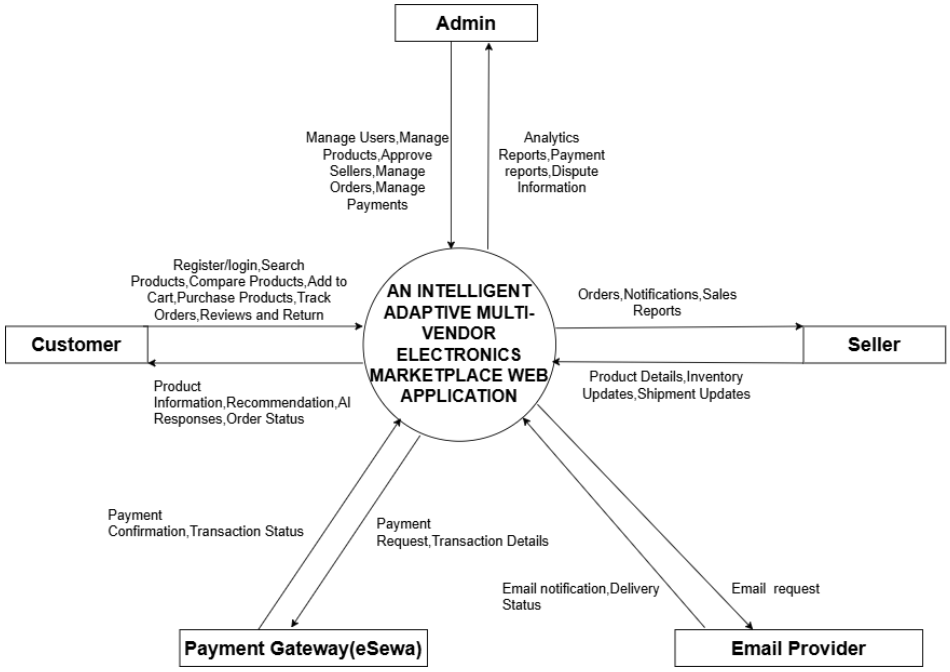

Figure 3.1: DFD Level 0

The DFD Level 0 provides a high-level view of the system, showing the three primary external entities (Customer, Seller, Administrator) and the single main process representing the entire marketplace system. Data flows shown include product browsing requests and recommendation responses to the Customer, product listing submissions from the Seller, and administrative control inputs from the Administrator.

3.3.3 Data Flow Diagram Level 1

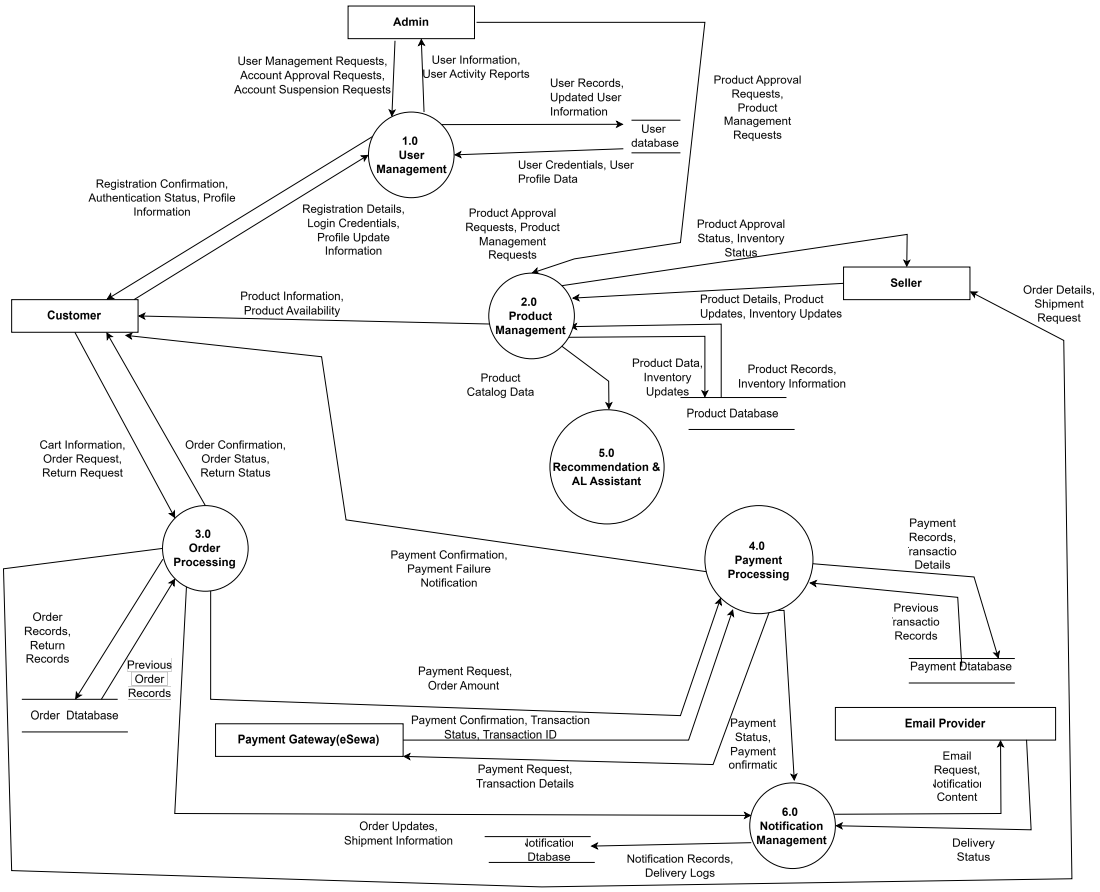

Figure 3.2: DFD Level 1

The DFD Level 1 decomposes the main system process into five principal sub-processes: User Management, Product Management, Order Management, Search and Recommendation, and Payment Processing. Data stores include the User Database, Product Catalog, Order Records, and Review Store. This level illustrates how data flows between sub-processes and which stores each process reads from and writes to.

Figure 3.3: Use Case Diagram

The Use Case Diagram illustrates the interactions between the three primary actors,i.e Customer, Seller, Administrator and the system's functional features. The Customer can register, log in, browse and search products, apply filters, upload an image for visual search, view product details, manage a cart and wishlist, place and track orders, write reviews, receive AI-powered recommendations, and interact with the conversational shopping assistant. The Seller can register and undergo verification, manage product listings, update order fulfillment status, and view sales analytics. The Administrator can approve or reject seller applications, manage all users, products, categories, and orders, and monitor platform activity through the administrative dashboard.

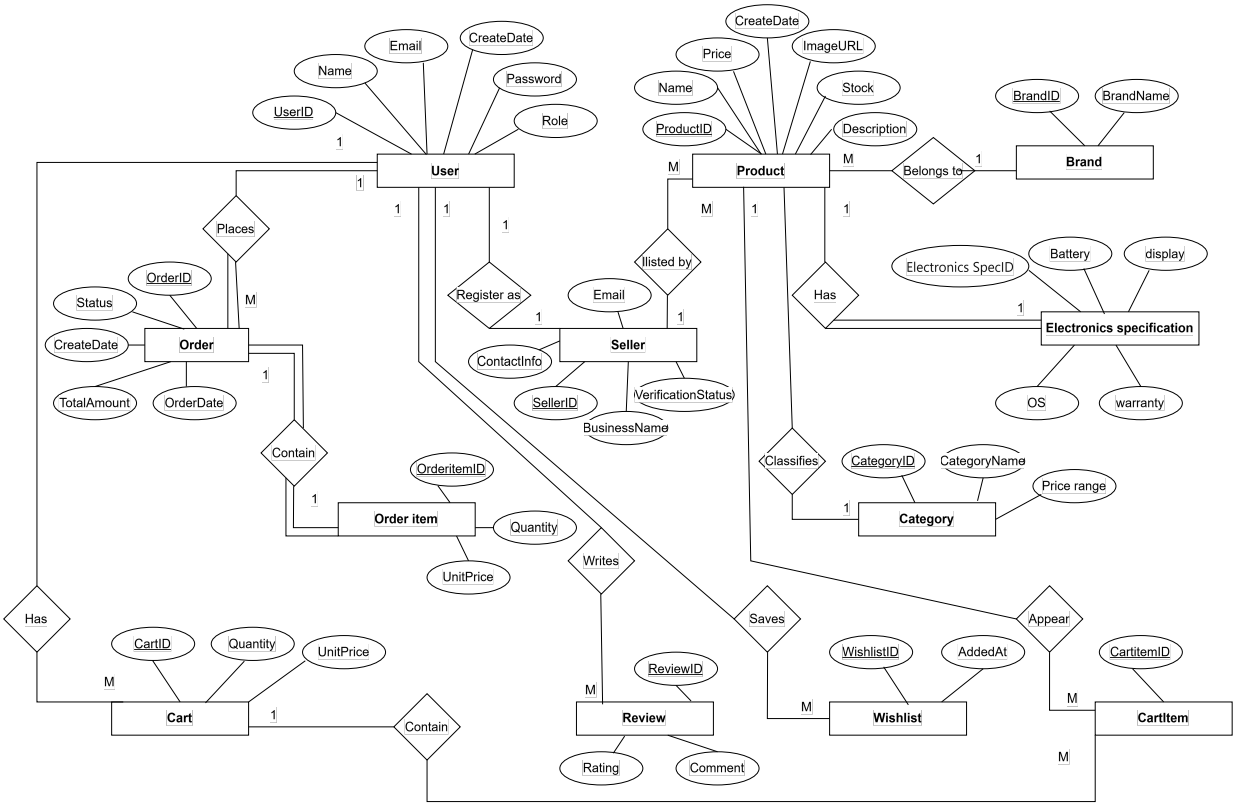

Figure 3.4: Entity Relationship Diagram

The ER diagram shows the relationships between the primary entities of the system: User, Seller, Product, Brand, Category, Product Specification, Order, Order Item, Cart, Review, and Wishlist. Key relationships include: a User places many Orders; a User registers as a Seller; a Seller lists many Products; a Product belongs to a Brand and is classified by a Category; an Order contains many Order Items; a User writes Reviews and saves Wishlists.

3.3.4 Schema Diagram

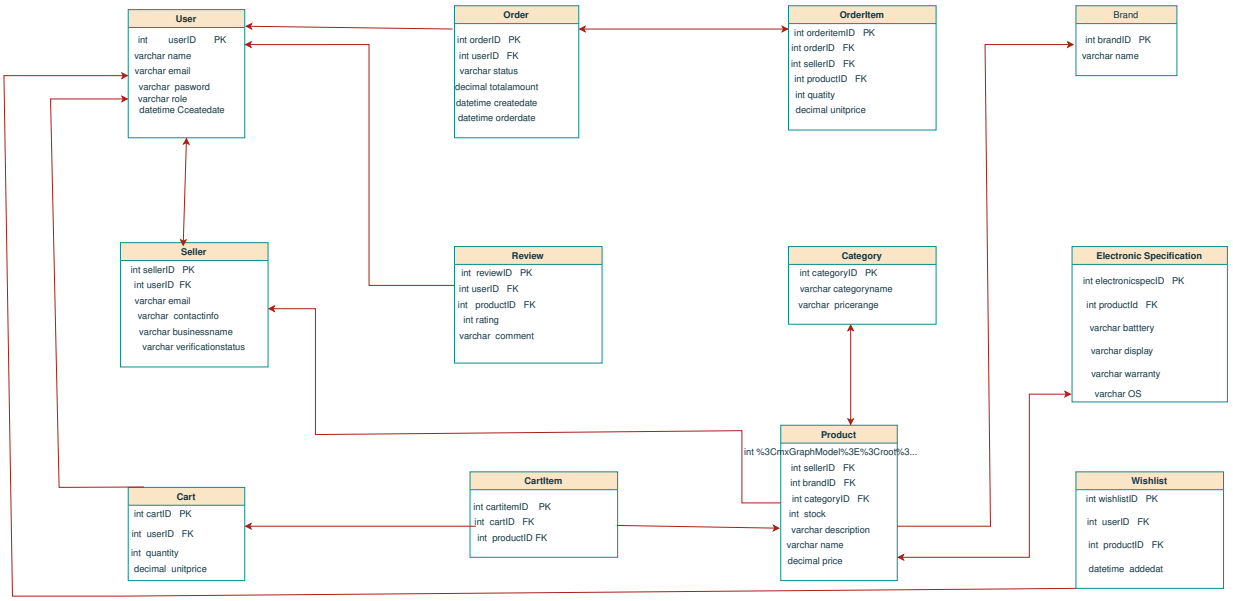

Figure 3.5: Schema Diagram

The schema diagram presents the relational database structure derived from the ER diagram, showing the concrete MySQL table definitions, column names, data types, primary keys, and foreign key constraints. The principal tables are: `users` (UserID PK, Name, Email, Password, Role), `sellers` (SellerID PK, UserID FK, BusinessName, VerificationStatus), `products` (ProductID PK, SellerID FK, BrandID FK, CategoryID FK, Name, Price, Stock, Description, ImageURL, AverageRating), `product_specifications` (SpecID PK, ProductID FK, Processor, RAM, Storage, Display, Battery, OS, Warranty), `categories` (CategoryID PK, CategoryName), `brands` (BrandID PK, BrandName), `orders` (OrderID PK, UserID FK, OrderDate, TotalAmount, Status), `order_items` (OrderItemID PK, OrderID FK, ProductID FK, Quantity, UnitPrice), `cart` (CartID PK, UserID FK), `cart_items` (CartItemID PK, CartID FK, ProductID FK, Quantity), `reviews` (ReviewID PK, UserID FK, ProductID FK, Rating, Comment), and `wishlist` (WishlistID PK, UserID FK, ProductID FK, AddedAt).

3.3.5 System Flowcharts

Two flowcharts are presented to illustrate the operational logic of the system's intelligent components: the Recommendation Engine and the Conversational AI Assistant.

Recommendation Engine Flow

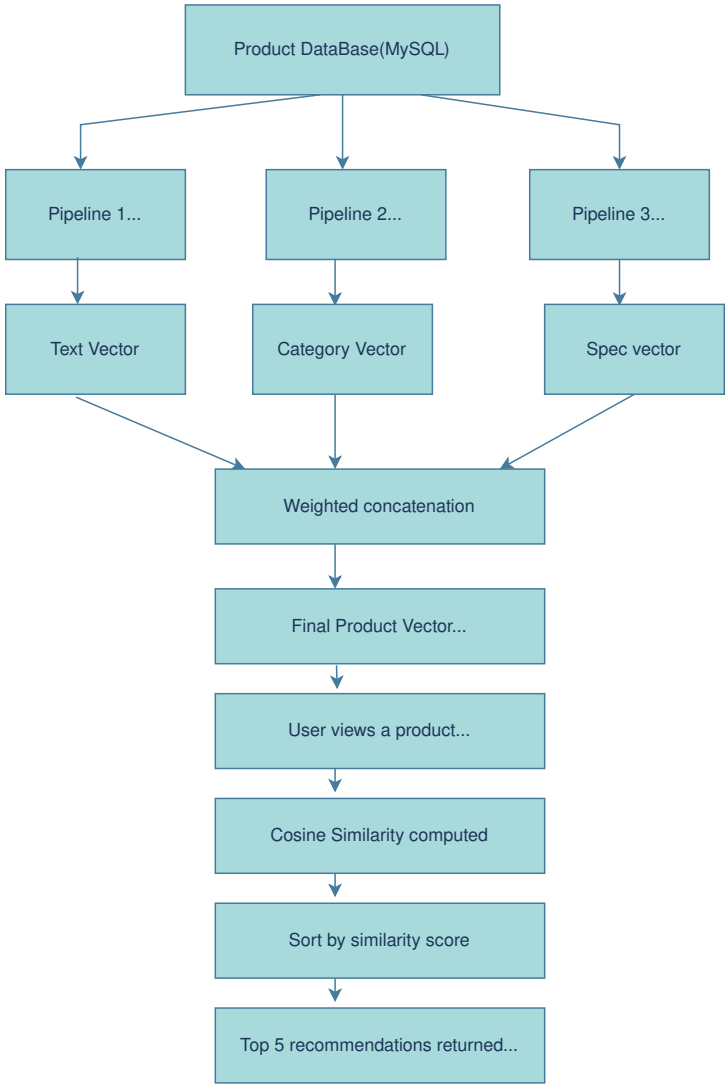

Figure 3.6: Recommendation Engine Flow Diagram

Figure 3.6 illustrates the workflow of the recommendation engine. The process begins when a user interacts with products by viewing, searching, or purchasing items. Product information such as category, brand, specifications, and textual descriptions is collected from the product catalog. The system then performs feature extraction using TF-IDF for textual attributes and one-hot encoding for categorical attributes. These features are combined into a unified product vector representation through weighted concatenation. Cosine similarity is subsequently calculated between products to identify items with similar characteristics. Based on the similarity scores, the recommendation engine retrieves the most relevant products and presents personalized recommendations to the user on product detail pages and throughout the shopping experience.

Conversational AI Assistant Flow

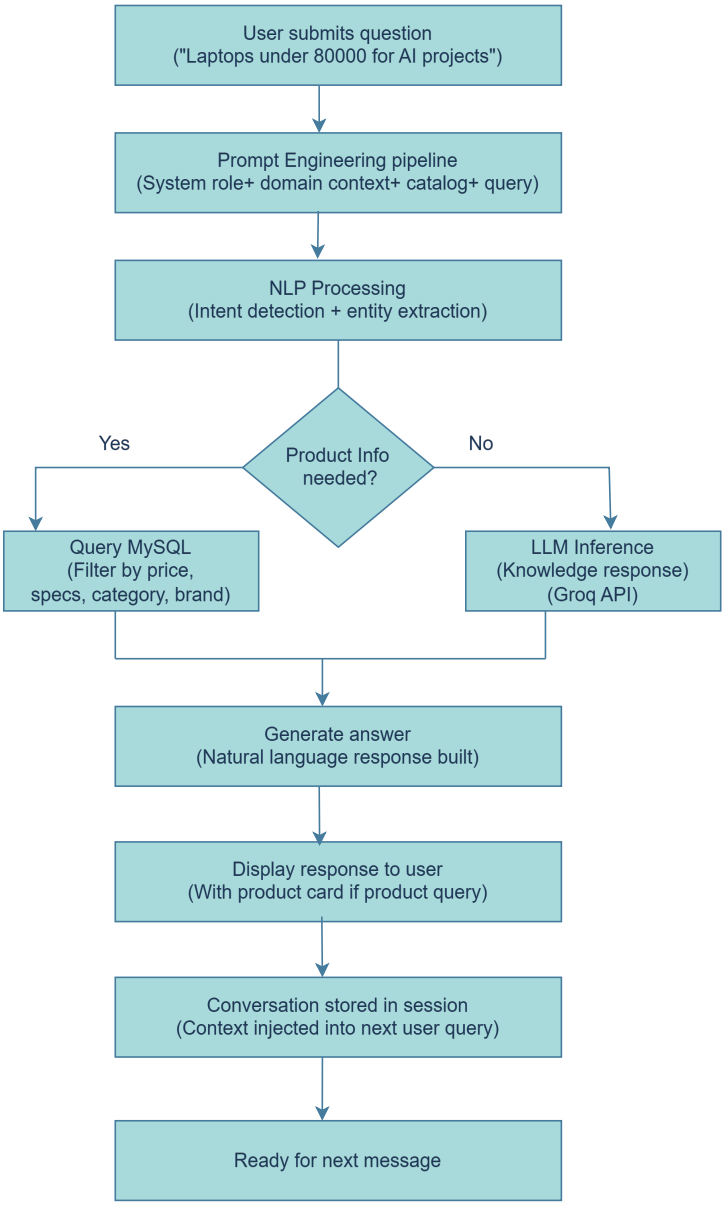

Figure 3.7: Conversational AI Assistant Flow Diagram

Figure 3.7 illustrates the processing workflow of the AI-powered shopping assistant. The process begins when a user submits a query through the chat interface. The query is forwarded to the backend, where prompt construction and context preparation are performed. Relevant product information is retrieved from the database whenever product-specific knowledge is required and incorporated into the prompt. The enriched prompt is then sent to the Large Language Model (LLM), which analyzes the user's intent and generates a context-aware response. The generated answer is returned to the application and displayed to the user. When appropriate, the response may also include

relevant product information and recommendations, enabling users to receive intelligent shopping assistance and product guidance.

3.3.6 Algorithm: Cosine Similarity for Product Recommendation

Purpose and Application

The Cosine Similarity algorithm is the core mathematical technique used in the Recommendation Engine of the proposed system. Its purpose is to measure the degree of similarity between any two products based on their attribute vectors, and to rank all products in the catalog by decreasing similarity to a product the user is currently viewing. The result is a list of the top-N most similar products, presented to the user as personalized recommendations on the product detail page and the homepage.

This algorithm is applied specifically at the point when a user views a product detail page. The system retrieves the viewed product's attribute vector, computes its cosine similarity against every other product vector in the catalog, sorts the results, and returns the top five most similar products as recommendations.

Why Cosine Similarity

Cosine Similarity is chosen over other recommendation algorithms for three reasons specific to this project. First, it addresses the cold-start problem: unlike collaborative filtering, which requires a large corpus of historical user interaction data to produce meaningful results, cosine similarity operates entirely on product attributes and requires no prior user history. This makes it suitable for a newly launched platform [7]. Second, it is computationally lightweight: for a focused electronics catalog, computing pairwise cosine similarities using Scikit-learn's `linear_kernel` over a TF-IDF or numerical attribute matrix is efficient and requires no GPU or training pipeline. Third, it is interpretable: the similarity score directly reflects how alike two products are in terms of category, brand, price range, and specifications, making the recommendations transparent and auditable.

How It Works

Each product in the catalog is represented as a feature vector $\mathbf{v} \in \mathbb{R}^n$, constructed from its attributes: category (one-hot encoded), brand (one-hot encoded), normalized price, and a TF-IDF weighted text representation of its name and description. Given two

product vectors $\mathbf{A}$ and $\mathbf{B}$, the cosine similarity is defined as:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \tag{3.1}$$

where $\mathbf{A} \cdot \mathbf{B}$ is the dot product of the two vectors, $\|\mathbf{A}\|$ is the Euclidean norm of $\mathbf{A}$, and $\|\mathbf{B}\|$ is the Euclidean norm of $\mathbf{B}$. The resulting score lies in the range $[0, 1]$: a score of 1 indicates the two products are identical in attribute space; a score of 0 indicates they share no common attributes.

The full recommendation procedure is described in Algorithm 1.

Algorithm 1 Content-Based Product Recommendation using Cosine Similarity
Require: Product catalog $\mathcal{P} = \{p_1, p_2, \ldots, p_m\}$, viewed product p_q
Ensure: Ordered list of top-N recommended products

Step 1: Feature Vector Construction
for each product $p_i \in \mathcal{P}$ do
    Encode category and brand as one-hot vectors
    Normalize price to $[0, 1]$ using min-max scaling
    Compute TF-IDF vector from product name and description
    Concatenate all features: $\mathbf{v}_i \leftarrow [\text{category} \| \text{brand} \| \text{price} \| \text{TF-IDF}]$
end for
Step 2: Similarity Computation
Retrieve feature vector $\mathbf{v}_q$ for the viewed product p_q
for each product $p_i \in \mathcal{P}$, $p_i \neq p_q$ do
    $s_i \leftarrow \dfrac{\mathbf{v}_q \cdot \mathbf{v}_i}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_i\|}$
end for
Step 3: Ranking and Selection
Sort all products by s_i in descending order
Return top-N products (default $N = 5$) as recommendations

Implementation

In the implementation, the feature matrix is pre-computed over the full product catalog at server startup and cached in memory using Scikit-learn's `TfidfVectorizer` and `linear_kernel` functions. When a recommendation request is received for product p_q, the system looks up the pre-computed similarity row for p_q, sorts the scores, filters out

out-of-stock items, and returns the top-N product IDs. These IDs are used to query the MySQL database for full product details, which are serialized and returned to the React frontend as a JSON response.

CHAPTER 4:

METHODOLOGY

4.1 Basic System Architecture

The proposed system follows a three-tier client-server architecture. The presentation tier consists of a React.js single-page application (SPA) that communicates with the backend exclusively through RESTful HTTP API calls. The application tier is implemented using Django REST Framework, which handles business logic, authentication, recommendation computation, and third-party API integration. The data tier consists of a MySQL relational database accessed through Django's Object-Relational Mapping (ORM) layer.

The AI Core comprising the Recommendation Engine, the Conversational Assistant, and the Visual Search Engine is integrated into the application tier as a set of service modules. The Recommendation Engine computes cosine similarity scores over product attribute vectors using Scikit-learn and is invoked by the DRF views serving the product detail and homepage endpoints. The Visual Search Engine uses OpenCV to extract and compare image feature vectors, enabling users to search for electronics products by uploading a photo. The Conversational Assistant is implemented as a custom prompt engineering pipeline: for each user query, the pipeline assembles a structured prompt consisting of a domain-specific system role, injected product catalog context, few-shot interaction examples, and the user's message, which is then forwarded to an LLM inference endpoint. The response is post-processed and returned to the frontend as a structured JSON object.

External services interface with the application tier via HTTPS: the LLM inference endpoint for conversational AI, the payment gateway (Stripe or Razorpay) for transaction processing, and an email service provider for transactional notifications.

4.2 Operational Block Diagram

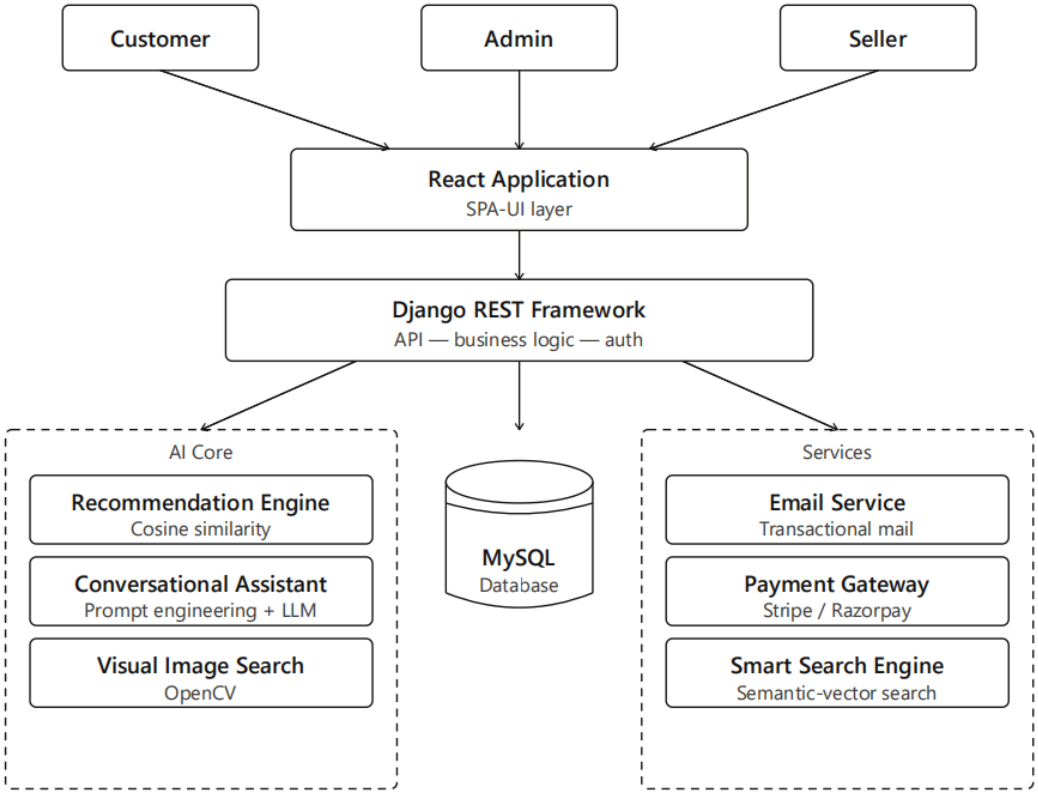

Figure 4.1: System Architecture Diagram

The operational flow of the system proceeds as follows. A user accesses the React.js frontend via a browser. Authentication requests are sent to the DRF backend, which validates credentials and issues JWT tokens. Authenticated requests to browse, search, or filter products are handled by the Product API, which queries the MySQL database and, where applicable, invokes the Recommendation Engine to append personalized suggestions to the response. Order placement triggers the Order Management module, which coordinates with the Payment Gateway and updates order status in the database. Conversational queries are routed to the Prompt Engineering Assistant module, which constructs a context-aware structured prompt and submits it to the LLM inference endpoint, returning the response to the frontend.

Figure 4.1 illustrates the complete system architecture. The three user roles (Customer, Admin, Seller) interact through the React Application SPA-UI layer. All requests pass through the Django REST Framework backend, which dispatches to three subsystems:

the AI Core (Recommendation Engine, Conversational Assistant, and Smart Search Engine), the MySQL database, and external services (Email Service and Payment Gateway).

4.3 Agile SCRUM Model

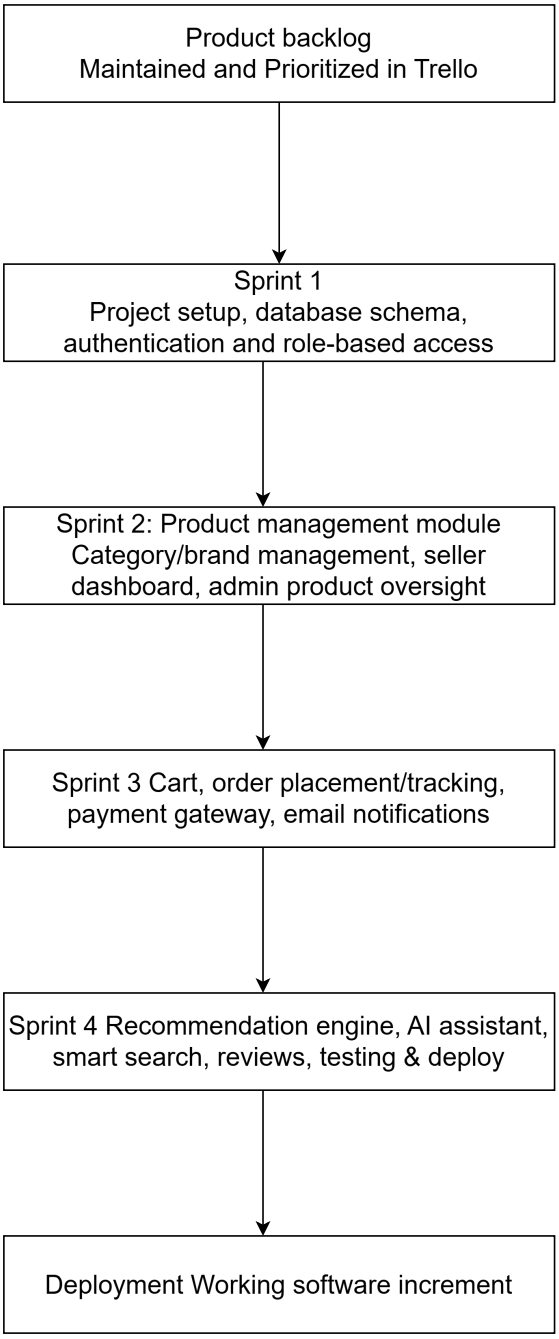

Figure 4.2: Agile Model Diagram

The project is developed using the Agile SCRUM methodology, organized into four two-week sprints. This iterative approach allows the team to deliver working software incrementally, incorporate feedback after each sprint review, and adapt to changing requirements throughout the development cycle.

- **Sprint 1:** Project setup, database schema design, user authentication (registration, login, JWT), and role-based access control for all three user roles.
- **Sprint 2:** Product management module (listing, editing, image upload), category and brand management, seller dashboard, and administrative product oversight.
- **Sprint 3:** Shopping cart, order placement and tracking, payment gateway integration, and email notification service.
- **Sprint 4:** Implementation of the recommendation engine, integration of conversational assistants with prompt engineering, smart search with price filtering, review system, and final testing and deployment.

Retrospectives and sprint reviews are conducted at the end of each sprint with the project supervisor. The product backlog is maintained and prioritized in Trello.

4.4 Dataset Description

The recommendation engine is built using a product dataset sourced from Kaggle, containing electronics product listings collected from online retail platforms. The dataset includes key product attributes required for content-based recommendation, such as product name, category, brand, price, and product description.

The dataset consists of approximately 500–1000 product entries, which is suitable for developing and evaluating a recommendation system for a small-to-medium-scale electronics marketplace targeting the Nepali market. The dataset serves as the initial product catalog during development and testing phases.

Prior to recommendation generation, the dataset undergoes preprocessing. Textual attributes such as product names and descriptions are cleaned and transformed into numerical representations using the TF-IDF (Term Frequency–Inverse Document Frequency) technique. Categorical attributes including brand and category are converted using one-hot encoding. Price values are normalized to ensure consistent feature scaling. The generated feature vectors are combined through weighted concatenation to create

a unified product representation. These vectors are then used by the cosine similarity recommendation engine to identify products with similar characteristics and generate personalized recommendations.

During deployment, the initial Kaggle dataset will be supplemented and gradually replaced by real product listings added by verified sellers through the platform, allowing the recommendation engine to operate on live marketplace data.

4.5 Tools Used

Category	**Technology**	**Purpose**
Frontend	React.js, Tailwind CSS	UI development
Backend	Django, Django REST Framework	API and business logic
Database	MySQL	Relational data storage
AI / ML	Scikit-learn, OpenCV	Recommendation engine, visual image search
Conversational AI	LLM inference endpoint, custom prompt engineering pipeline	Domain-specific shopping assistant
Authentication	JSON Web Tokens (JWT)	Secure session management
Payment	Stripe / Razorpay	Transaction processing
Version Control	Git, GitHub	Source code management
Project Management	Trello	SCRUM backlog and sprint tracking
IDE	VS Code	Development environment
API Testing	Postman	Endpoint testing and documentation

Table 4.1: Development Tools and Technologies

CHAPTER 5:

PROJECT SCHEDULING AND COST ESTIMATION

5.1 Gantt Chart

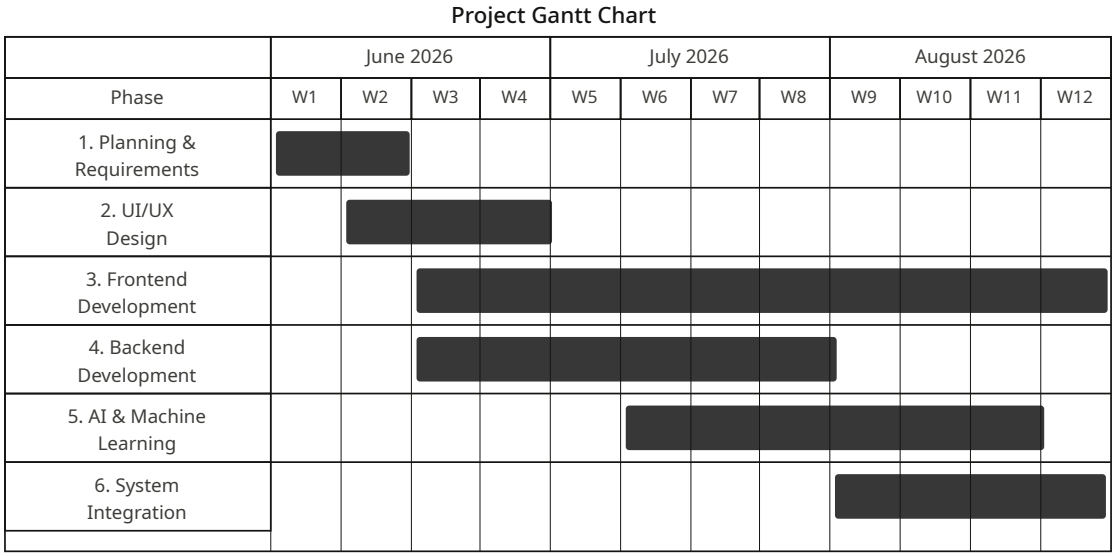

Figure 5.1: Project Timeline Gantt Chart

The project is planned over a 16-week period from June to September 2026. Major milestones include: requirements and design completion (weeks 1–2), Sprint 1 delivery (weeks 3–4), Sprint 2 delivery (weeks 5–8), Sprint 3 delivery (weeks 9–12), Sprint 4 and AI integration (weeks 13–15), and final testing and proposal submission (week 16).

5.2 Cost Estimation

Item	**Details**	**Estimated Cost (NPR)**
LLM Inference Usage	Free tier inference (e.g. Groq API free tier or locally hosted Ollama) for custom prompt-engineered conversational assistant	0
Cloud Hosting	Free tier deployment (e.g. Render, Railway, or Vercel for frontend)	0
Domain and SSL	.np domain (free via Mercantile Communications, Nepal); SSL via Let's Encrypt (free)	0
Internet and Utilities	College network and laboratory infrastructure	0

Table 5.1: Cost Estimation for Project Development

Total Estimated Cost: 0 NPR

All software frameworks, libraries, and development tools used are open-source and incur no licensing costs. The conversational assistant is built through a custom prompt engineering pipeline using a free-tier LLM inference endpoint, incurring zero API costs. Cloud deployment is handled through free-tier hosting services, and internet access is provided through college infrastructure. The only anticipated out-of-pocket expense is printing and documentation for the final submission.

CHAPTER 6:
EXPECTED OUTCOMES OR RESULTS

Upon successful completion of the project, the following outcomes are expected:

1. A fully functional multi-vendor electronics marketplace web application with role-based access for customers, sellers, and administrators, deployable on a cloud server.
2. A content-based recommendation engine that generates personalized product suggestions based on cosine similarity over product attribute vectors, demonstrating measurable improvement in product discovery compared to non-personalized browsing.
3. A custom conversational shopping assistant built through a systematic prompt engineering pipeline, capable of understanding natural language electronics product queries, extracting structured intent (product type, price range, brand preference), and returning relevant product suggestions — demonstrating that domain-specific AI behavior can be achieved through careful prompt design rather than model fine-tuning.
4. A unified search and filtering interface combining semantic search, price-range filtering, category-based navigation, and OpenCV-powered visual image search into a single discovery pipeline, enabling users to locate electronics products by uploading a photo.
5. A seller management and verification module enabling secure multi-vendor operations with administrative oversight.
6. A documented and tested RESTful API backend suitable for future extension, including potential mobile application clients.
7. A project report and proposal conforming to Tribhuvan University, Institute of Engineering formatting guidelines, with comprehensive system design documentation.

The system will be evaluated against defined functional test cases for all user roles,

non-functional benchmarks for response time and uptime, and qualitative usability assessment by a sample user group.

REFERENCES

[1] E. Turban, D. King, J. K. Lee, T.-P. Liang, and D. C. Turban, *Electronic Commerce: A Managerial and Social Networks Perspective*, 8th ed. New York, NY, USA: Springer, 2015.

[2] K. C. Laudon and C. G. Traver, *E-Commerce: Business, Technology, Society*, 17th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] L. Richardson, *RESTful Web APIs: Services for a Changing World*. Sebastopol, CA, USA: O'Reilly Media, 2018.

[4] M. Masse, *REST API Design Rulebook*. Sebastopol, CA, USA: O'Reilly Media, 2011.

[5] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, Jan.–Feb. 2003.

[6] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.

[7] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender systems survey," *Knowl.-Based Syst.*, vol. 46, pp. 109–132, Jul. 2013.

[8] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–38, Jan. 2019.

[9] B. A. Shawar and E. Atwell, "Chatbots: Are they really useful?" *LDV Forum*, vol. 22, no. 1, pp. 29–49, 2007.

[10] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, "Language models are few-shot learners," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[11] J. White, Q. Fu, S. Hays, J. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering

with ChatGPT," *arXiv preprint arXiv:2302.11382*, Feb. 2023. [Online]. Available: https://arxiv.org/abs/2302.11382

[12] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, Jan. 2023.

[13] L. Cui, S. Yang, F. Chen, Z. Wei, M. Wang, and D. Zhao, "MRC-based slot filling for spoken language understanding," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 3977–3987.

[14] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval: The Concepts and Technology behind Search*, 2nd ed. New York, NY, USA: ACM Press/Addison-Wesley, 2011.

[15] K.-P. Yee, K. Swearingen, K. Li, and M. Hearst, "Faceted metadata for image search and browsing," in *Proc. SIGCHI Conf. Human Factors Comput. Syst. (CHI)*, 2003, pp. 401–408.

[16] P. Morville and L. Rosenfeld, *Information Architecture for the World Wide Web*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2006.

[17] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA, USA: O'Reilly Media, 2008.